Leasibility AI — Layout Engine Interface Contract

Document type: Developer Interface Specification

Version: 1.0 — March 2026

Scope: `server/aiEngine.ts` — the `generateScenarios()` function and its `GeneratedScenario` return type

Purpose: Define the exact input and output contract so a developer can replace or improve the layout engine without breaking the reporting system or PDF pipeline.

Overview

The layout engine is the function `generateScenarios(input: ScenarioInput)` exported from `server/aiEngine.ts`. It is called once per project analysis and returns an array of exactly **three** `GeneratedScenario` objects. These three objects are written to the `scenarios` database table and subsequently consumed by the dashboard UI (`ProjectDetail.tsx`), the public share page (`SharedReport.tsx`), and the PDF export system (`pdfRouter.ts`).

A developer improving the layout engine must preserve the exact shape of both `ScenarioInput` and `GeneratedScenario`. The rest of the application depends on these types as a stable contract. No other file needs to change.

Part 1 — Input: `ScenarioInput`

The following object is assembled in `server/routers.ts` (the `projects.analyze` mutation, lines 233–246) and passed directly to `generateScenarios()`.

Full Input Type

Plain Text

```
interface ScenarioInput {
  propertyName:          string;          // Display name of the property
  totalSqFt:             number;          // Total rentable square footage of t
  headcount:             number;          // Number of people the tenant needs
  industry:              string;          // Tenant's industry (e.g. "Technolog
  market:                string;          // City or market name used for cost
  floorPlanUrl?:         string;          // CDN URL of the uploaded floor plan
  programNotes?:         string;          // Free-text notes from the broker (o
```

```
  // Custom room count mode — only present when programMode === "custom"
  programMode?:          "headcount" | "custom";
  customOpenWorkstations?: number;
  customPrivateOffices?:   number;
  customConferenceRooms?:  number;
  customHuddleRooms?:      number;
}
```

Field-by-Field Reference

Field	Type	Required	Source in DB	Notes
`propertyName`	`string`	Yes	`projects.propertyName`	Used in AI prompt and PDF cover page
`totalSqFt`	`number`	Yes	`projects.totalSqFt`	Drives all area calculations and cost benchmarks
`headcount`	`number`	Yes	`projects.headcount`	Used for sq ft/person ratio and room count formulas
`industry`	`string`	Yes	`projects.industry`	Informs room type mix in AI prompt
`market`	`string`	Yes	`projects.market ?? projects.city`	Selects the correct $/sqft benchmark from `MARKET_BENCHMARKS`
`floorPlanUrl`	`string | undefined`	No	`projects.floorPlanUrl`	If present, passed as `originalImages` to the image generation API. Switches the AI image prompt from "generate a floor plate" to

				"overlay this uploaded plan"
`programNotes`	`string | undefined`	No	`projects.programNotes` (cleaned)	Free-text injected into the AI prompt. The `[CUSTOM_PROGRAM_MODE]` machine-readable block is stripped before this field is populated
`programMode`	`"headcount" | "custom"`	No	Parsed from `projects.programNotes`	When `"custom"`, the AI prompt is prepended with exact room count requirements
`customOpenWorkstations`	`number`	No	Parsed from `programNotes`	Only used when `programMode === "custom"`
`customPrivateOffices`	`number`	No	Parsed from `programNotes`	Only used when `programMode === "custom"`
`customConferenceRooms`	`number`	No	Parsed from `programNotes`	Only used when `programMode === "custom"`
`customHuddleRooms`	`number`	No	Parsed from `programNotes`	Only used when `programMode === "custom"`

Scenario Definitions Passed to the Engine

The three scenarios are fixed by the application. The layout engine must always return exactly three objects in this order:

Index	`scenarioNumber`	`impactLevel`	`label`	Design Intent

0	1	`"low"`	`"Light Refresh"`	Preserve all existing walls; cosmetic changes only
1	2	`"medium"`	`"Moderate Build-Out"`	Selective demolition; reuse MEP where possible
2	3	`"high"`	`"Full Transformation"`	Complete gut and rebuild; no existing constraints

---

Part 2 — Output: `GeneratedScenario`

The function must return `Promise<GeneratedScenario[]>` — an array of exactly three objects. Each object is written to one row in the `scenarios` database table by `server/routers.ts`.

Full Output Type

Plain Text

```
interface GeneratedScenario {
  // Identity
  scenarioNumber:     number;                        // 1, 2, or 3 — fixed by i
  label:              string;                        // "Light Refresh" | "Mode
  impactLevel:        "low" | "medium" | "high";    // fixed by index
  impactTag:          string;                        // same as label — display

  // Space metrics
  efficiencyScore:    number;                        // 0–100 integer (usable/t
  usableSqFt:         number;                        // integer — totalSqFt × (
  totalSqFt:          number;                        // integer — same as input

  // Room program
  roomBreakdown:      Array<{
    type:   string;   // room type name (e.g. "Open Workspace", "Conference Roo
    count:  number;   // number of this room type in the plan
    sqFt:   number;   // area per unit in square feet
  }>;

  // Layout representation
```

```
  layoutDescription:  string;   // 2–3 sentence narrative of the spatial strate
  layoutSvg:          string;   // inline SVG string — deterministic fallback f
  layoutImageUrl?:    string;   // CDN URL of AI-generated architectural image

  // Budget — all values in USD
  budgetLow:          number;   // total project cost at low benchmark ($/sqft
  budgetMid:          number;   // total project cost at mid benchmark
  budgetHigh:         number;   // total project cost at high benchmark
  costPerSqFtLow:     number;   // low $/sqft benchmark for this market and imp
  costPerSqFtMid:     number;   // mid $/sqft benchmark
  costPerSqFtHigh:    number;   // high $/sqft benchmark
  budgetBreakdown: {
    construction: { low: number; mid: number; high: number };  // 60% of total
    ffe:          { low: number; mid: number; high: number };  // 18% — Furnitu
    itAv:         { low: number; mid: number; high: number };  // 10% — IT / Au
    softCosts:    { low: number; mid: number; high: number };  // 8%  — Design,
    tiAllowance:  { low: number; mid: number; high: number };  // 4%  — TI Allo
  };

  // Schedule
  scheduleWeeksLow:   number;   // fastest realistic timeline (weeks)
  scheduleWeeksMid:   number;   // most likely timeline (weeks)
  scheduleWeeksHigh:  number;   // conservative timeline (weeks)
  schedulePhases:     Array<{
    phase:       string;   // phase name (e.g. "Design & Permitting")
    weeks:       string;   // duration as string (e.g. "3–5" or "2")
    description: string;   // one-sentence description of work in this phase
  }>;

  // Narrative
  aiSummary:          string;   // 3–4 sentences written directly to the tenant
}
```

Field-by-Field Reference

The table below maps each output field to the database column that stores it and the downstream system that reads it.

Output Field	DB Column (`scenarios` table)	Consumed by PDF	Consumed by Dashboard UI
`scenarioNumber`	`scenarioNumber` (int)	Scenario header label	Tab index
`label`	`label` (varchar 100)	Scenario header	Tab label

`impactLevel`	`impactLevel` (enum)	Color coding (green/gold/red)	Color coding
`impactTag`	— (not stored separately)	—	—
`efficiencyScore`	`efficiencyScore` (int)	Key metric card	Efficiency badge
`usableSqFt`	`usableSqFt` (int)	Key metric card	Usable sq ft display
`totalSqFt`	`totalSqFt` (int)	—	—
`roomBreakdown`	`roomBreakdown` (json)	Program Breakdown list	Room breakdown table
`layoutDescription`	`layoutDescription` (text)	—	Layout description text
`layoutSvg`	`layoutSvg` (text)	SVG fallback if no image	SVG fallback
`layoutImageUrl`	`layoutImageUrl` (text)	**Primary floor plan image**	**Primary floor plan image**
`budgetLow`	`budgetLow` (float)	Budget range display	Budget range card
`budgetMid`	`budgetMid` (float)	—	Mid budget display
`budgetHigh`	`budgetHigh` (float)	Budget range display	Budget range card
`costPerSqFtLow`	`costPerSqFtLow` (float)	—	Cost/sqft display
`costPerSqFtMid`	`costPerSqFtMid` (float)	—	Cost/sqft display
`costPerSqFtHigh`	`costPerSqFtHigh` (float)	—	Cost/sqft display
`budgetBreakdown`	`budgetBreakdown` (json)	Budget Breakdown table	Budget breakdown table
`scheduleWeeksLow`	`scheduleWeeksLow` (int)	Timeline range footer	Timeline display
`scheduleWeeksMid`	`scheduleWeeksMid` (int)	Key metric card ("X wks")	Timeline card
`scheduleWeeksHigh`	`scheduleWeeksHigh` (int)	Timeline range footer	Timeline display

`schedulePhases`	`schedulePhases` (json)	Project Schedule table	Schedule phases
`aiSummary`	`aiSummary` (text)	AI Analysis section	AI summary card

Part 3 — How Output Flows into the Reporting System

3.1 Database Write (routers.ts)

Immediately after `generateScenarios()` returns, `server/routers.ts` calls `createScenario()` once for each of the three objects. Every field in `GeneratedScenario` maps to a column in the `scenarios` table. The `impactTag` field is the only field that is **not** persisted — it is used only during the generation pass and is not needed downstream.

3.2 Dashboard UI (ProjectDetail.tsx)

When a broker opens a completed project, the frontend calls `trpc.projects.get.useQuery({ id })`, which returns the project row and all three scenario rows from the database. The UI renders a tabbed interface — one tab per scenario — displaying the following fields in order:

1. **Efficiency score** and **usable sq ft** as headline metrics.
2. **`layoutImageUrl`** as the primary floor plan image. If `layoutImageUrl` is `null` or `undefined`, the UI falls back to rendering the **`layoutSvg`** string inline.
3. **`aiSummary`** as the AI Analysis narrative card.
4. **`roomBreakdown`** as a two-column list of space types with counts and areas.
5. **Budget range** (`budgetLow` – `budgetHigh`) and **cost per sq ft** as a metric card.
6. **`budgetBreakdown`** as a categorised table (Construction, FF&E, IT/AV, Soft Costs, TI Allowance).
7. **`scheduleWeeksMid`** as the headline timeline figure.
8. **`schedulePhases`** as a phased project schedule table.

3.3 Public Share Page (SharedReport.tsx)

The share page is a public, unauthenticated view of the same data. It reads the same scenario fields via a share token lookup and renders them in the same structure as the dashboard. The `layoutImageUrl` / `layoutSvg` fallback logic is identical.

3.4 PDF Export (pdfRouter.ts)

The PDF router (`server/pdfRouter.ts`) fetches the project and all three scenario rows from the database and passes them to `buildReportHtml()` . This function renders a multi-page HTML document that is stored in S3 and returned to the browser for printing. The following fields are consumed by the PDF system:

PDF Section	Fields Used
Cover page	`project.propertyName` , `project.propertyAddress` , `project.floorNumber` , `project.totalSqFt` , `project.headcount` , `project.market` , `broker.*`
Scenario header	`scenarioNumber` , `impactLevel` (for color), `label`
Key metrics row	`efficiencyScore` , `usableSqFt` , `budgetLow` , `budgetHigh` , `scheduleWeeksMid`
Floor plan section	`layoutImageUrl` (preferred) → `layoutSvg` (fallback)
AI Analysis	`aiSummary`
Program Breakdown	`roomBreakdown[].type` , `roomBreakdown[].count` , `roomBreakdown[].sqFt`
Budget Breakdown	`budgetBreakdown.construction/ffe/itAv/softCosts/tiAllowance` × `{low, mid, high}`
Project Schedule	`schedulePhases[].phase` , `schedulePhases[].weeks` , `schedulePhases[].description` , `scheduleWeeksLow` , `scheduleWeeksHigh`

The PDF system has **no dependency** on `layoutDescription` , `impactTag` , `costPerSqFtLow/Mid/High` , or the individual `budgetLow/Mid/High` totals (it reads the breakdown categories instead).

Part 4 — Constraints the Layout Engine Must Respect

A developer replacing the layout engine must satisfy the following constraints to maintain compatibility with the full pipeline.

Array length. The function must return exactly three `GeneratedScenario` objects. The application does not handle zero, one, two, or four scenarios.

Index-to-impact mapping. Index 0 must have `impactLevel: "low"`, index 1 must have `impactLevel: "medium"`, and index 2 must have `impactLevel: "high"`. The cost benchmarks, schedule benchmarks, and color coding in the UI and PDF are all keyed to `impactLevel`, not to `label`.

`efficiencyScore` range. The value must be an integer between 0 and 100. The current benchmarks are: Low = 72–80, Medium = 80–87, High = 85–93. The UI renders this as a percentage badge; values outside 0–100 will display incorrectly.

`roomBreakdown` structure. Each item must have `type` (string), `count` (integer ⩾ 1), and `sqFt` (integer > 0). The PDF renders these as `{count}x {type} ({sqFt} sq ft)`. The `type` string is also used as a key in the SVG renderer's color map.

`layoutSvg` must be a valid inline SVG string. It is injected directly into the HTML of the PDF and the dashboard without sanitisation. It must begin with `<svg` and end with `</svg>`.

`layoutImageUrl` must be a publicly accessible HTTPS URL or `undefined`. The PDF embeds it as an `<img src="...">` tag. An empty string or a relative path will produce a broken image in the PDF.

`budgetBreakdown` must be an object with exactly five keys (`construction`, `ffe`, `itAv`, `softCosts`, `tiAllowance`), each containing `{ low, mid, high }` number values. The PDF iterates `Object.entries(budgetBreakdown)` and maps each key to a human-readable label. Additional or renamed keys will appear with their raw key name in the PDF.

`schedulePhases` must be an array of objects with `phase` (string), `weeks` (string or number), and `description` (string). The PDF renders `{weeks} wks` — if `weeks` is a string like `"3–5"`, it renders as `"3–5 wks"`, which is the intended format.

`aiSummary` must be plain text. It is injected into an HTML `<p>` tag in the PDF without markdown rendering. Markdown syntax will appear as raw characters.

Part 5 — What the Layout Engine Does NOT Need to Produce

The following data is computed **outside** the layout engine, in `generateScenarios()` itself, after the layout engine returns. A developer replacing only the layout/room-placement logic does not need to produce these values — they are derived automatically:

- All budget figures (`budgetLow`, `budgetMid`, `budgetHigh`, `costPerSqFt*`, `budgetBreakdown`) — computed from `MARKET_BENCHMARKS` using `market` and `impactLevel`.
- All schedule figures (`scheduleWeeksLow/Mid/High`, `schedulePhases`) — computed from `SCHEDULE_BENCHMARKS` using `impactLevel`.
- `usableSqFt` — computed as `Math.round(totalSqFt × (efficiencyScore / 100))`.

- `layoutSvg` — generated by `generateSVGLayout()` from `roomBreakdown` and `impactLevel`.
- `layoutImageUrl` — generated by the AI image generation API using `roomBreakdown` and `floorPlanUrl`.

The minimum viable output from a replacement layout engine is therefore:

Plain Text

```
{
  label:             string;   // one of the three fixed labels
  efficiencyScore:   number;   // 0–100
  roomBreakdown:     Array<{ type: string; count: number; sqFt: number }>;
  layoutDescription: string;
  aiSummary:         string;
}
```

Everything else in `GeneratedScenario` is computed from these five fields plus the input parameters.

Part 6 — Integration Diagram

Plain Text

```
NewProject.tsx / ProjectDetail.tsx (frontend)
    │
    │  trpc.projects.analyze.mutate({ projectId })
    ▼
server/routers.ts — projects.analyze procedure
    │
    │  Assembles ScenarioInput from DB project row
    │  Parses programMode / custom room counts from programNotes
    │
    ▼
server/aiEngine.ts — generateScenarios(ScenarioInput)
    │
    ├── [1] invokeLLM() → roomBreakdown, efficiencyScore, layoutDescription, ai
    ├── [2] generateSVGLayout() → layoutSvg  (deterministic, from roomBreakdown
    ├── [3] generateImage() → layoutImageUrl  (AI image, may fail → undefined)
    ├── [4] MARKET_BENCHMARKS lookup → budget figures
    └── [5] SCHEDULE_BENCHMARKS lookup → schedule figures
    │
    │  Returns GeneratedScenario[3]
    ▼
server/routers.ts
    │
```

```
    │  createScenario() × 3  →  scenarios table (MySQL)
    ▼
    ┌──────────────────────────────────────────────────────────┐
    │                   scenarios table                        │
    │  (all GeneratedScenario fields stored as columns/json)   │
    └──────────────────────────────────────────────────────────┘
         │                    │                    │
         ▼                    ▼                    ▼
  ProjectDetail.tsx    SharedReport.tsx      pdfRouter.ts
  (dashboard UI)       (public share)        (PDF export)
         │                    │                    │
         └────────────────────┴────────────────────┘
                              │
                        Browser / PDF
```

Summary

A developer improving the layout engine should focus on the five fields that the engine itself must produce: `label`, `efficiencyScore`, `roomBreakdown`, `layoutDescription`, and `aiSummary`. All financial, schedule, and visual rendering data is derived automatically from these fields and the input parameters. The stable public API is the `generateScenarios(input: ScenarioInput): Promise<GeneratedScenario[]>` function signature — its input and output types must not change. Only the internal implementation may be replaced.